

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки і комп'ютерних технологій

Звіт
про виконання лабораторної роботи №1
з курсу «Системи штучного інтелекту»
на тему:
**«Сліпий пошук на графах. Особливості реалізації пошуку в ширину у
графах»**

Виконав:
студент групи ФЕІ-32
Трофимов Володимир
Перевірів:
доц. Любунь З.М.

Зміст

Тема	3
Мета	3
Вхідні дані	3
Завдання	3
Критерії виконання роботи	3
Теоретичні відомості	4
1. Графи та їх основні поняття	4
2. Основні види графів	5
2.1. Неорієнтований граф	5
2.2. Орієнтований граф	5
3. Дерево.....	5
4. Пошук у графах	6
5. Пошук у ширину	6
6. Черга у BFS	7
7. Алгоритм BFS	7
8. Відновлення знайденого шляху	8
9. BFS та найкоротший шлях	8
Інтерфейс та принцип роботи програми	9
Результати дослідження роботи програми	13
<i>Дослідження 1: зміна напрямку обходу вершин на неорієнтованому графі.</i>	13
<i>Дослідження 2: зміна початку та кінця пошуку місцями.</i>	15
<i>Дослідження 3: Видалення та додавання вершин, ребер та дуг, які входять у вирішальний граф.</i> ..	15
<i>Дослід 4: Використання пошуку вшир на різних видах графів.</i>	17
Висновок	21
Код програми	22

Тема

Сліпий пошук на графах. Особливості реалізації пошуку в ширину у графах.

Мета

Створити програму, яка реалізує пошук в ширину на графі, та дослідити особливості її роботи на графах різних видів та при різних умовах пошуку.

Вхідні дані

- звичайний розгалужений граф (не менше 5-и гілок) порядку не менше 30 та розміром не менше 30-40 (не менше 5-и “гілок” у випадку дерева; граф повинен бути заданий явно, з зафіксованим положенням (координатами) вершин; рандомне задання графа не допускається)
- початкова і цільова вершини графу;
- порядок обходу вершин графу при пошуку.

Завдання

Створити програму виконання пошуку в ширину на одній з мов програмування високого рівня, в якій:

- візуально представити заданий граф;
- реалізувати знаходження шляху між заданими вершинами графу, використовуючи алгоритм пошуку в ширину;
- дослідити вплив зміни порядку і розмірів графу, а також заміни звичайного графу на оргграф на результати пошуку;
- відмітити переваги і недоліки цього виду пошуку.

Критерії виконання роботи

1. Створити графічний інтерфейс керування роботою програми, який має передбачати можливість проводити з його використанням всі потрібні для виконання завдання операції, у т. ч.:

- візуалізацію графу, самого процесу пошуку і представлення його результатів та часу виконання пошуку;
- можливість змінювати розмірність (введення і видалення вершин) і порядок (додавання і видалення дуг і ребер) графу, а також виду графу (звичайного, оргграфу та дерева);
- можливість змінювати напрям обходу вершин графу при пошуку; – представлення результатів пошуку (знайдений шлях на графі при заданих вершинах, кількість використаних при його знаходженні циклів та кількість розкритих вершин), які виводяться в окремому вікні (вікнах) інтерфейсу.

2. Дослідити особливості роботи написаної програми та результати пошуку, отримані при:

- різних напрямках обходу суміжних вершин при розкритті вершини;
- зміні напрямку пошуку (при дзеркальній заміні початкової вершини у і мети пошуку);
- видалення та додавання вершин, ребер та дуг, які входять у вирішальний граф;
- вплив на результати пошуку виду графа – дослідити особливості пошуку вшир на дереві, звичайному та орієнтованому графах однакового порядку та розміру і різних розмірів, а також заміни деяких ребер графа на однонапрямлені дуги.

Теоретичні відомості

1. Графи та їх основні поняття

Граф — математична структура, що складається з множини вершин та множини зв'язків між ними. Граф позначається:

$$G = (V, E),$$

де:

- V — множина вершин графа;
- E — множина ребер або дуг, що з'єднують вершини.

Вершина є елементом графа, який може представляти деякий об'єкт або стан системи. *Ребро* задає зв'язок між двома вершинами.

Наприклад:

$$V = \{v_1, v_2, v_3, v_4\},$$

$$E = \{(v_1, v_2), (v_1, v_3), (v_2, v_4)\}.$$

Граф може задаватися за допомогою списків суміжності, матриці суміжності або графічно.

Порядок та розмір графа

Порядком графа називається кількість його вершин:

$$|V| = n.$$

Розміром графа називається кількість його ребер або дуг:

$$|E| = m.$$

Таким чином, для графа з 30 вершинами та 40 ребрами:

$$n = 30, m = 40.$$

2. Основні види графів

2.1. Неорієнтований граф

У неорієнтованому графі ребро не має напрямку.

Якщо існує ребро:

$$(v_1, v_2),$$

то з вершини v_1 можна перейти до v_2 , а з v_2 — до v_1 .

Графічно таке ребро зображається звичайною лінією:

$$v_1 - v_2$$

Для BFS це означає, що при розкритті вершини необхідно враховувати всі вершини, з'єднані з нею ребрами.

2.2. Орієнтований граф

Орієнтований граф, або *орграф*, — це граф, у якому кожен зв'язок має певний напрямок.

Зв'язок між вершинами називається *дугою*:

$$v_1 \rightarrow v_2.$$

У цьому випадку перехід з v_1 до v_2 дозволений, а з v_2 до v_1 — не обов'язково.

Наприклад:

$$v_1 \rightarrow v_2 \rightarrow v_3.$$

Під час BFS напрямки дуг необхідно враховувати. Якщо існує лише:

$$v_1 \rightarrow v_2,$$

то під час пошуку від v_2 до v_1 цей зв'язок використовуватися не може.

Саме тому заміна частини ребер неорієнтованого графа на односпрямовані дуги може призвести до:

- зміни знайденого шляху;
- збільшення довжини шляху;
- відсутності шляху;
- зміни кількості розкритих вершин.

3. Дерево

Дерево — це зв'язний неорієнтований граф, у якому між будь-якими двома вершинами існує рівно один простий шлях.

Для дерева з n вершинами кількість ребер завжди дорівнює:

$$m = n - 1.$$

Наприклад, для 30 вершин дерево має:

$$m = 29.$$

Основними властивостями дерева є:

- відсутність циклів;
- зв'язність;
- між двома будь-якими вершинами існує єдиний шлях;
- додавання нового ребра між уже існуючими вершинами створює цикл.

Для пошуку в ширину дерево є особливо зручним випадком, оскільки між початковою та цільовою вершинами існує тільки один шлях. Тому зміна порядку обходу суміжних вершин не змінює сам шлях, хоча може змінювати порядок розкриття вершин.

4. Пошук у графах

Пошук у графі — це процес систематичного переходу від одних вершин графа до інших для знаходження вершини, яка задовольняє задану умову.

Зазвичай задаються:

- початкова вершина S ;
- цільова вершина G ;
- граф $G = (V, E)$;
- порядок розгляду суміжних вершин.

У лабораторній роботі необхідно знайти шлях:

$$S \rightarrow \dots \rightarrow G.$$

Існують різні алгоритми пошуку, зокрема:

- пошук у ширину (*Breadth-First Search, BFS*);
- пошук у глибину (*Depth-First Search, DFS*);
- пошук з обмеженням глибини;
- пошук з ітеративним поглибленням;
- евристичний пошук тощо.

У цій лабораторній роботі використовується пошук у ширину.

5. Пошук у ширину

Пошук у ширину (*Breadth-First Search, BFS*) — алгоритм обходу графа, який досліджує вершини пошарово: спочатку початкову вершину, потім усі вершини на відстані одного ребра від неї, потім на відстані двох ребер і т. д.

Принцип роботи можна представити так:

→ вершини першого рівня
→ вершини другого рівня
→ вершини третього рівня
→ ...

Тобто BFS спочатку досліджує найближчі до початкової вершини вершини, і лише після цього переходить до віддаленіших.

6. Черга у BFS

Основною структурою даних алгоритму BFS є черга (Queue).

Черга працює за принципом FIFO (First In, First Out) тобто першим доданий елемент буде першим вилучений.

Наприклад:

Додали: $A \rightarrow B \rightarrow C \rightarrow D$

Черга: $[A, B, C, D]$

Вилучаємо A: $[B, C, D]$

Вилучаємо B: $[C, D]$

Саме використання FIFO забезпечує пошаровий характер BFS.

7. Алгоритм BFS

Нехай задано:

- S — початкова вершина;
- T — цільова вершина.

Основний алгоритм можна описати так:

1. Створити порожню чергу.
2. Додати початкову вершину S до черги.
3. Позначити S як відвідану.
4. Поки черга не порожня:
 - вилучити першу вершину з черги;
 - перевірити, чи є вона цільовою;
 - переглянути її суміжні вершини;
 - невідвідані вершини додати до черги;
 - позначити їх як відвідані.
5. Якщо цільову вершину знайдено — відновити шлях.
6. Якщо черга спорожня, а ціль не знайдено — шляху не існує.

8. Відновлення знайденого шляху

Сам BFS знаходить цільову вершину, але для виведення *повного шляху* необхідно запам'ятовувати попередника кожної відвіданої вершини.

Наприклад:

$$A \rightarrow B \rightarrow D \rightarrow G$$

Під час пошуку можна отримати:

$$\text{parent}[B] = A$$

$$\text{parent}[D] = B$$

$$\text{parent}[G] = D$$

Після знаходження G шлях відновлюється у зворотному напрямку:

$$G \leftarrow D \leftarrow B \leftarrow A$$

Після розвороту:

$$A \rightarrow B \rightarrow D \rightarrow G$$

9. BFS та найкоротший шлях

Однією з найважливіших властивостей BFS є те, що для *невагового графа* він знаходить найкоротший шлях за кількістю ребер.

Нехай існують шляхи:

$$A \rightarrow B \rightarrow G$$

Та

$$A \rightarrow C \rightarrow D \rightarrow E \rightarrow G$$

BFS знайде:

$$A \rightarrow B \rightarrow G$$

оскільки його довжина становить 2 ребра, тоді як другого 4.

Отже:

$$dBFS(A, G) = mind(A, G),$$

де $d(A, G)$ — кількість ребер у шляху між вершинами.

Важливо: BFS гарантує найкоротший шлях саме за кількістю ребер. Якщо ребра мають різну вартість, звичайний BFS не гарантує найменшу сумарну вартість.

Інтерфейс та принцип роботи програми

Графічний інтерфейс розробленої програми реалізовано мовою Python з використанням бібліотеки CustomTkinter, яка забезпечує створення сучасних елементів графічного інтерфейсу та їх взаємодію з алгоритмічною частиною програми. Структуру інтерфейсу організовано у вигляді трьох основних функціональних областей: панелі навігації між вкладками, панелі керування параметрами графа та алгоритму BFS і області візуалізації графа та процесу пошуку.

Основна функціональність програми зосереджена у вкладці «Лабораторна», яка призначена для виконання експериментів та дослідження особливостей алгоритму пошуку в ширину. Додатково передбачено вкладки «Результати» та «Теорія BFS».

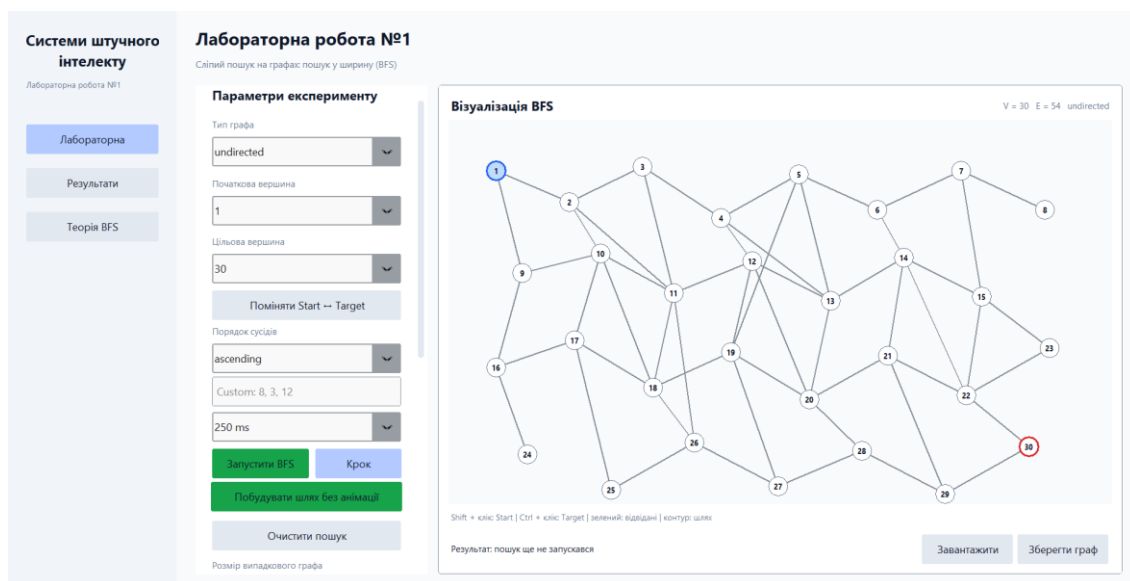


Рис.1 – Загальний вигляд інтерфейсу.

Вкладка «Лабораторна» є основним робочим середовищем програми та містить засоби налаштування параметрів експерименту, керування графом, запуску алгоритму пошуку та його візуалізації.

У секції «Параметри експерименту» користувач може задавати основні параметри пошуку та структури графа. Передбачено можливість вибору типу графа: неорієнтований граф, орієнтований граф або дерево. Це дозволяє досліджувати вплив структури та напрямленості зв'язків на результати роботи алгоритму BFS.

Також задаються початкова (Start) та цільова (Target) вершини пошуку. Для проведення експериментів зі зміною напрямку пошуку передбачено функцію «Поміняти Start ↔ Target», яка автоматично змінює початкову та цільову вершини місцями.

Окремим параметром є порядок обходу суміжних вершин. Програма підтримує декілька режимів: обхід у зростаючому порядку, у спадаючому порядку та за спеціально заданою користувачем послідовністю. Даний параметр використовується для дослідження впливу порядку розгляду суміжних вершин на процес BFS та вибір конкретного шляху у випадку існування декількох найкоротших маршрутів.

Для керування швидкістю демонстрації алгоритму передбачено параметр часу виконання одного кроку. Він використовується під час покрокової анімації пошуку та дозволяє спостерігати послідовність розкриття вершин і зміни стану графа.

Основні операції керування пошуком представлені такими функціями:

- «Запустити BFS» — виконує пошук у ширину від заданої початкової вершини до цільової;
- «Крок» — виконує один етап алгоритму, що дає змогу детально простежити послідовність роботи BFS;
- «Побудувати шлях без анімації» — виконує пошук та відображає кінцевий результат без покрокової візуалізації;
- «Очистити пошук» — видаляє результати поточного пошуку та повертає граф до початкового стану.

Параметри експерименту

Тип графа
undirected

Початкова вершина
1

Цільова вершина
30

Поміняти Start ↔ Target

Порядок сусідів
ascending
Custom: 8, 3, 12

250 ms

Запустити BFS Крок

Побудувати шлях без анімації

Очистити пошук

Рис.2 –Вигляд (основної частини) параметрів експерименту.

Для дослідження різних конфігурацій графа реалізовано секцію «Редагування графа». За її допомогою можна змінювати структуру графа шляхом

додавання та видалення вершин, ребер і дуг. Такий функціонал дозволяє досліджувати вплив зміни порядку та розміру графа на результати пошуку.

Крім того, передбачено можливість генерування випадкового графа. Дана функція має допоміжний характер і використовується для додаткового тестування працездатності програмної реалізації. Основний граф, використаний для виконання лабораторного дослідження, задається явно із заздалегідь визначеними вершинами, ребрами та координатами їх розташування, що відповідає вимогам лабораторної роботи.

Для швидкого повернення до початкової конфігурації передбачено функцію «Reset Graph». Вона відновлює вихідну структуру графа після виконання експериментів з додаванням або видаленням його елементів.

Центральну частину робочого вікна займає область візуалізації графа. Вершини графа відображаються у фіксованих координатах, а зв'язки між ними представлені відповідними ребрами або напрямленими дугами. Для орієнтованих графів напрямок переходу додатково позначається стрілками.

Під час виконання BFS візуально відображаються проміжні стани алгоритму. Це дозволяє простежити порядок розкриття вершин, формування черги та поступове наближення до цільової вершини. Зміна візуального стану вершин використовується для розмежування невідвіданих, відвіданих, поточної та розкритої вершин. Після успішного завершення пошуку знайдений маршрут додатково виділяється на графі.

Такий підхід забезпечує наочне представлення алгоритму та дозволяє безпосередньо співвіднести теоретичний принцип пошуку в ширину з фактичною послідовністю обробки вершин.

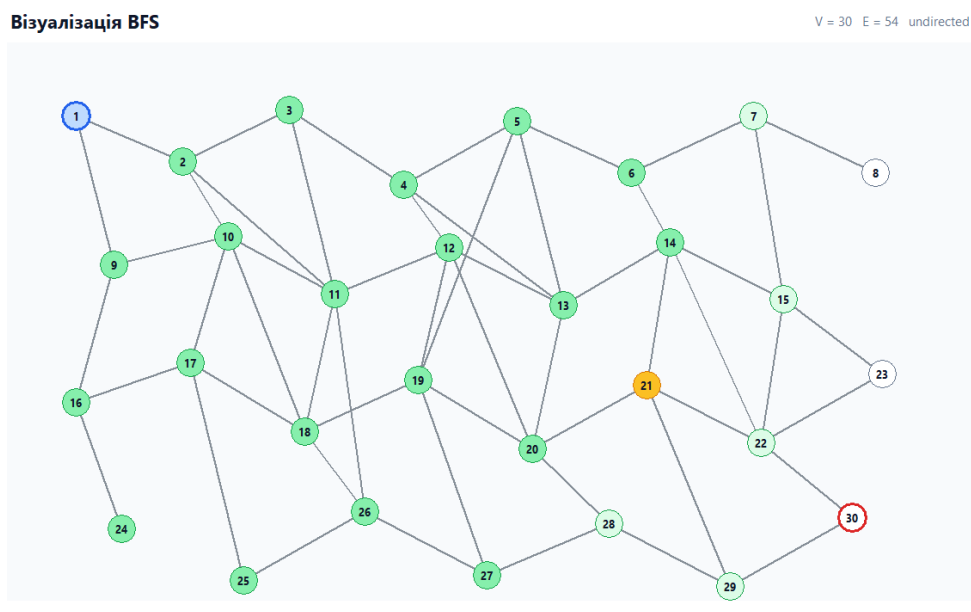


Рис.3 – Візуалізація графа

Вкладка «Результати» призначена для фіксації результатів проведених експериментів. Результати можуть зберігатися у вигляді окремих записів, що дозволяє порівнювати різні конфігурації графа та параметри пошуку.

Для кожного експерименту можуть фіксуватися основні характеристики виконання BFS, зокрема початкова та цільова вершини, тип графа, порядок обходу суміжних вершин, знайдений шлях, його довжина, кількість розкритих вершин, кількість циклічних переходів та час виконання алгоритму.

Додавання запису до таблиці результатів здійснюється користувачем після завершення відповідного експерименту. Це дозволяє зберігати лише ті результати, які безпосередньо використовуються під час подальшого аналізу.

Результати та експерименти										
Зберігайте результати запуску для порівняння типів графіє і порядку обходу.										
Додати поточний результат Очистити таблицю										
№	Type	V	E	Start	Target	Order	Path	Expanded	Iterations	Time
1	undirected	30	54	1	30	ascending	7	30	30	7594.051 ms

Рис.4 – Вкладка «Результати»

Вкладка «Теорія BFS» містить стислий довідковий матеріал щодо алгоритму пошуку в ширину. У ній наведено основний принцип роботи BFS, особливості використання черги, порядок розкриття вершин, умови знаходження найкоротшого шляху та основні характеристики алгоритму.

Наявність теоретичного розділу безпосередньо в програмному середовищі забезпечує можливість оперативного звернення до необхідних теоретичних відомостей під час проведення експериментів.

Теорія: пошук у ширину
BFS досліджує граф рівень за рівнем, використовуючи чергу FIFO. Алгоритм: 1. Помістити початкову вершину в чергу та позначити її як віддану. 2. Вилучити першу вершину з черги. 3. Додати всіх невідданих сусідів і запам'ятати їхнього батька (попередника). 4. Зупинитися, коли знайдено цільову вершину або черга порожня. Часова складність: $O(V + E)$ Просторова складність: $O(V)$ Переваги: повний для скінченних графів; знаходить найкоротший шлях у незваженому графі; систематичний та ефективний, коли ціль розташована близько. Недоліки: може споживати значний обсяг пам'яті, розгортати зайві вершини, ігнорує ваги ребер і сповільнюється на графах із великим коефіцієнтом розгалуження.

Рис.5 – Вкладка «Теорія BFS»

Таким чином, розроблений графічний інтерфейс об'єднує засоби формування та редагування графа, налаштування параметрів пошуку, покрокового виконання BFS, візуального спостереження за його роботою та фіксації отриманих результатів. Це забезпечує необхідну функціональність для проведення досліджень впливу структури графа, його розміру, порядку обходу суміжних вершин та напрямку пошуку на результати роботи алгоритму пошуку в ширину.

Результати дослідження роботи програми

Дослідження 1: зміна напрямку обходу вершин на неорієнтованому графі.

Перший етапом буде обхід графа за зростанням.

Візуалізація BFS

V = 30 E = 54 undirected

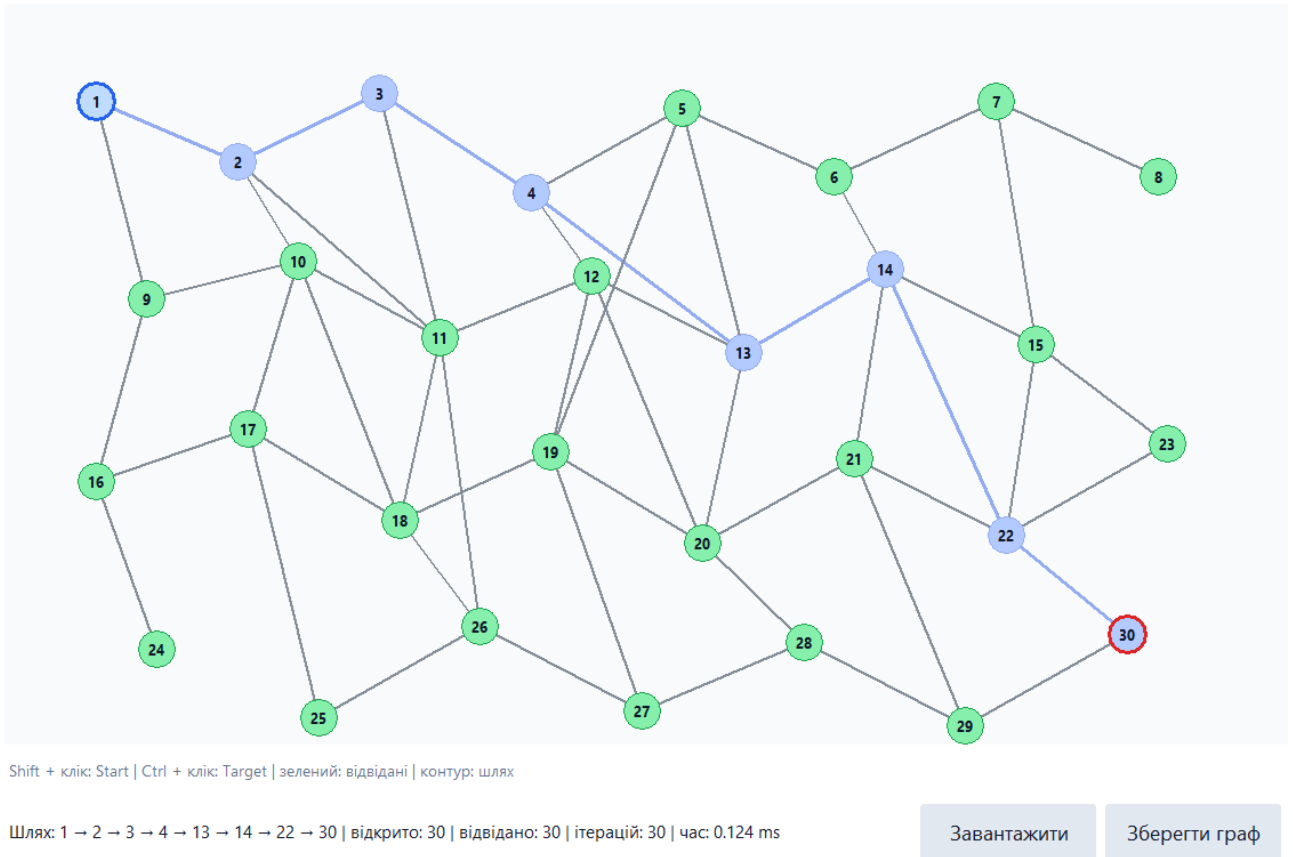
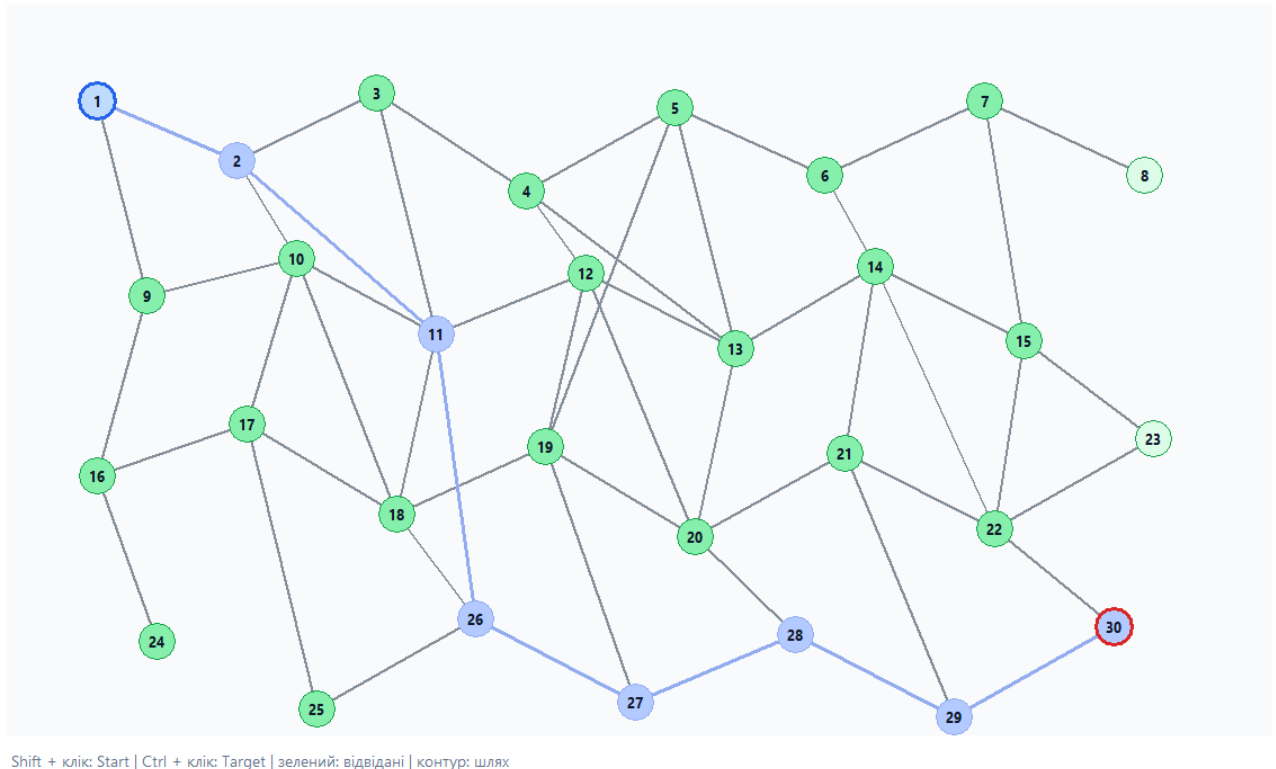


Рис.6 – Обхід неорієнтованого графа за зростанням

Згідно рисунка 6, даний обхід показав такі результати: кількість відкритих вершин 30, відвіданих: 30, що вказує на повний обхід графа для побудови найкоротшого маршруту.

Тепер використаємо обхід за спаданням та власний обхід (цей обхід використовує обхід за зростанням та додатково наші правила обходу).

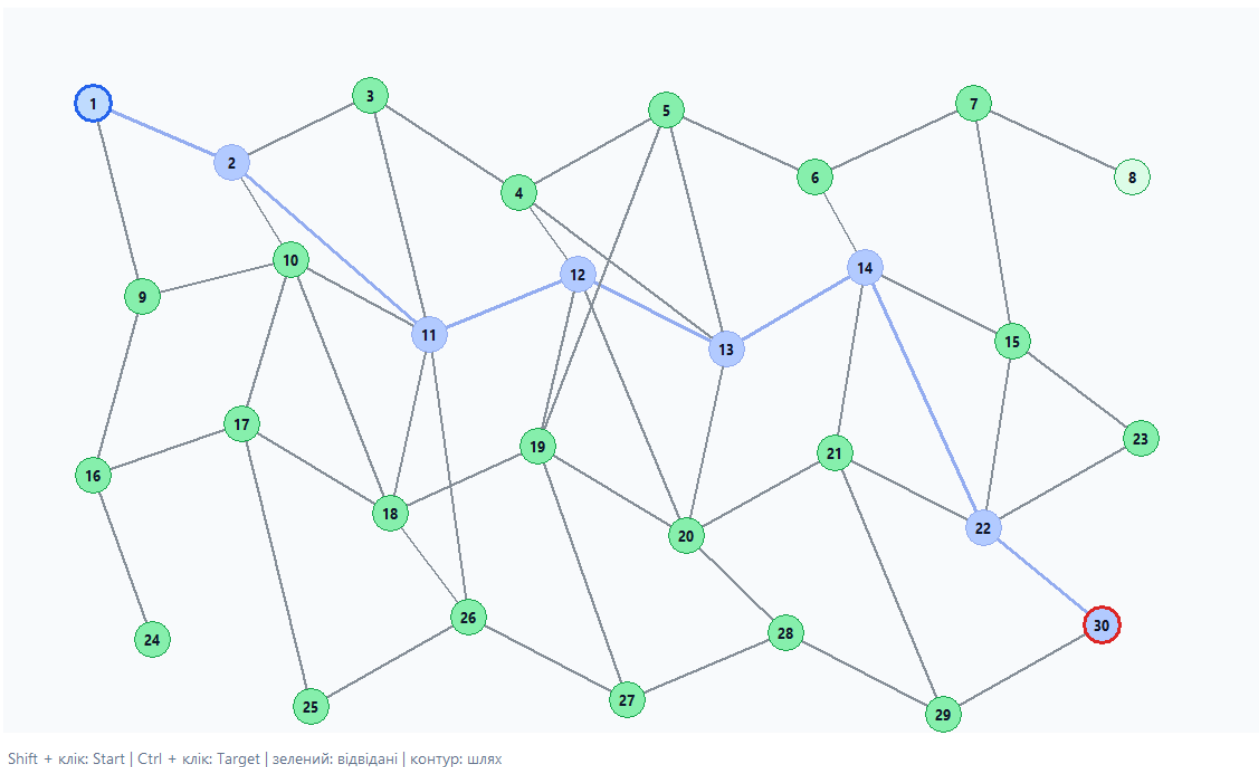


Шлях: 1 → 2 → 11 → 26 → 27 → 28 → 29 → 30 | відкрито: 28 | відвідано: 30 | ітерацій: 28 | час: 0.264 ms

Завантажити

Зберегти граф

Рис.7 – Обхід неорієнтованого графа за спаданням



Шлях: 1 → 2 → 11 → 12 → 13 → 14 → 22 → 30 | відкрито: 29 | відвідано: 30 | ітерацій: 29 | час: 0.387 ms

Завантажити

Зберегти граф

Рис.8 – Обхід неорієнтованого графа за власним порядком (в даному випадку створено правило: 11 має більший пріоритет ніж 3).

Як видно на рисунку 7 та рисунку 8 результати змінилися для даного графу. В цьому експерименті для даного графу найкраще показав себе обхід за спаданням.

Дослідження 2: зміна початку та кінця пошуку місцями.

Виконаємо даний експеримент.

Візуалізація BFS

V = 30 E = 54 undirected

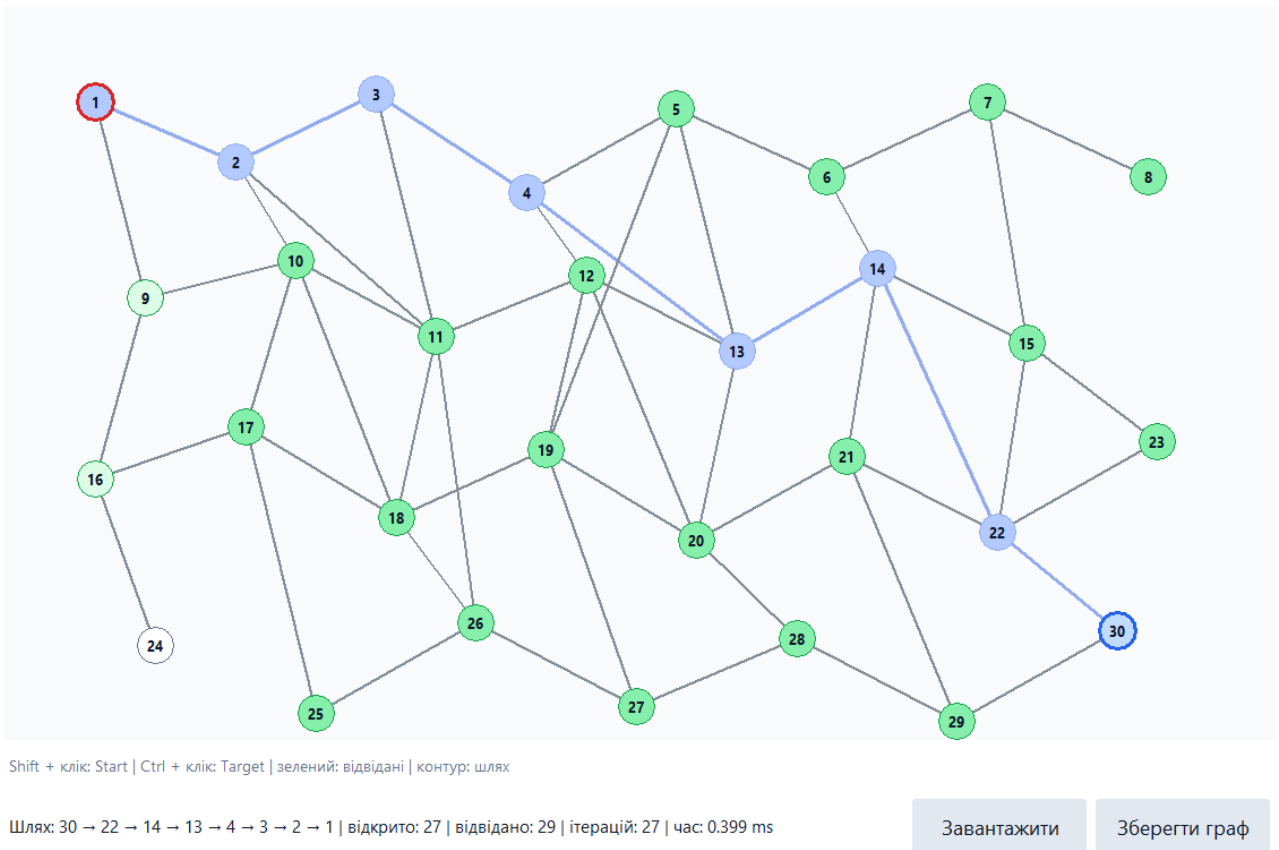
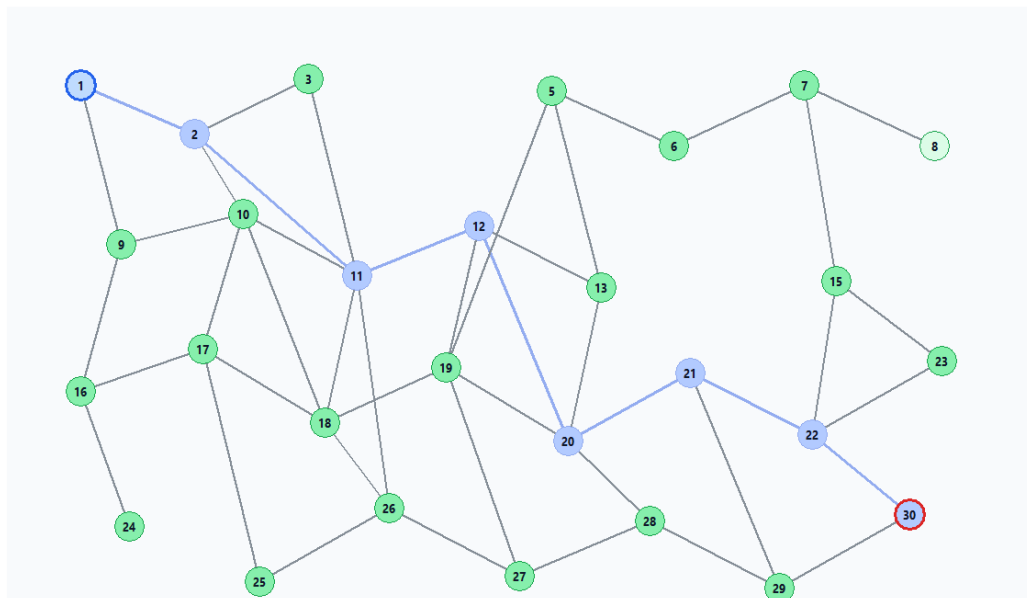


Рис.9 – Обхід неорієнтованого графа змінивши початку та кінцеві вершину місцями.

Для даного графу краще згідно рисунка 9 краще використовувати шлях від вершини 30 до вершини 1, оскільки задіюється менше пам'яті для пошуку шляху.

Дослідження 3: Видалення та додавання вершин, ребер та дуг, які входять у вирішальний граф.

Проведемо даний дослід та видалимо вершини 4 та 14.



Shift + клік: Start | Ctrl + клік: Target | зелений: відвідані | контур: шлях

Шлях: 1 → 2 → 11 → 12 → 20 → 21 → 22 → 30 | відкрито: 27 | відвідано: 28 | ітерацій: 27 | час: 0.249 ms

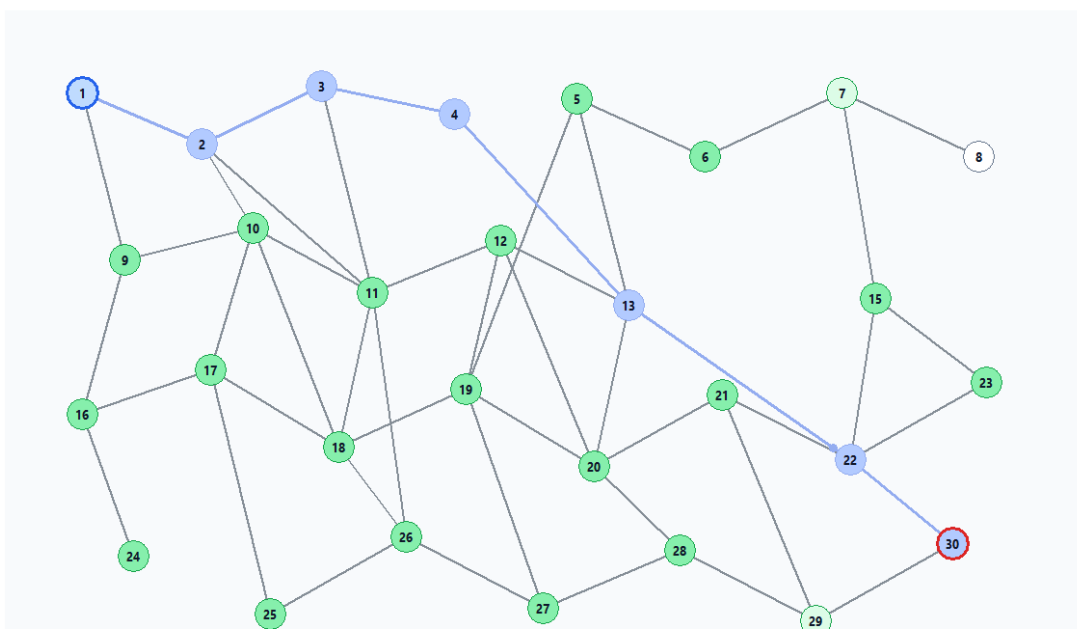
Завантажити

Зберегти граф

Рис.10 – Обхід неорієнтованого графа видаливши вершини 4 та 14.

Результат показав зміну шляху, однак тепер граф не повністю побував у всіх точках, що вказує на зменшення кількості ітерацій для пошуку маршруту.

Тепер додамо назад вершину 4 яка буде поєднана із вершиною 13 та 3. Також додамо дугу від 13 до 22.



Shift + клік: Start | Ctrl + клік: Target | зелений: відвідані | контур: шлях

Шлях: 1 → 2 → 3 → 4 → 13 → 22 → 30 | відкрито: 26 | відвідано: 28 | ітерацій: 26 | час: 0.310 ms

Завантажити

Зберегти граф

Рис.11 – Обхід неорієнтованого графа додавши вершину 4, ребра та дуги.

На рисунку 11 видно, що шлях використовує додану дугу. Тому змінимо початок і кінець місцями.

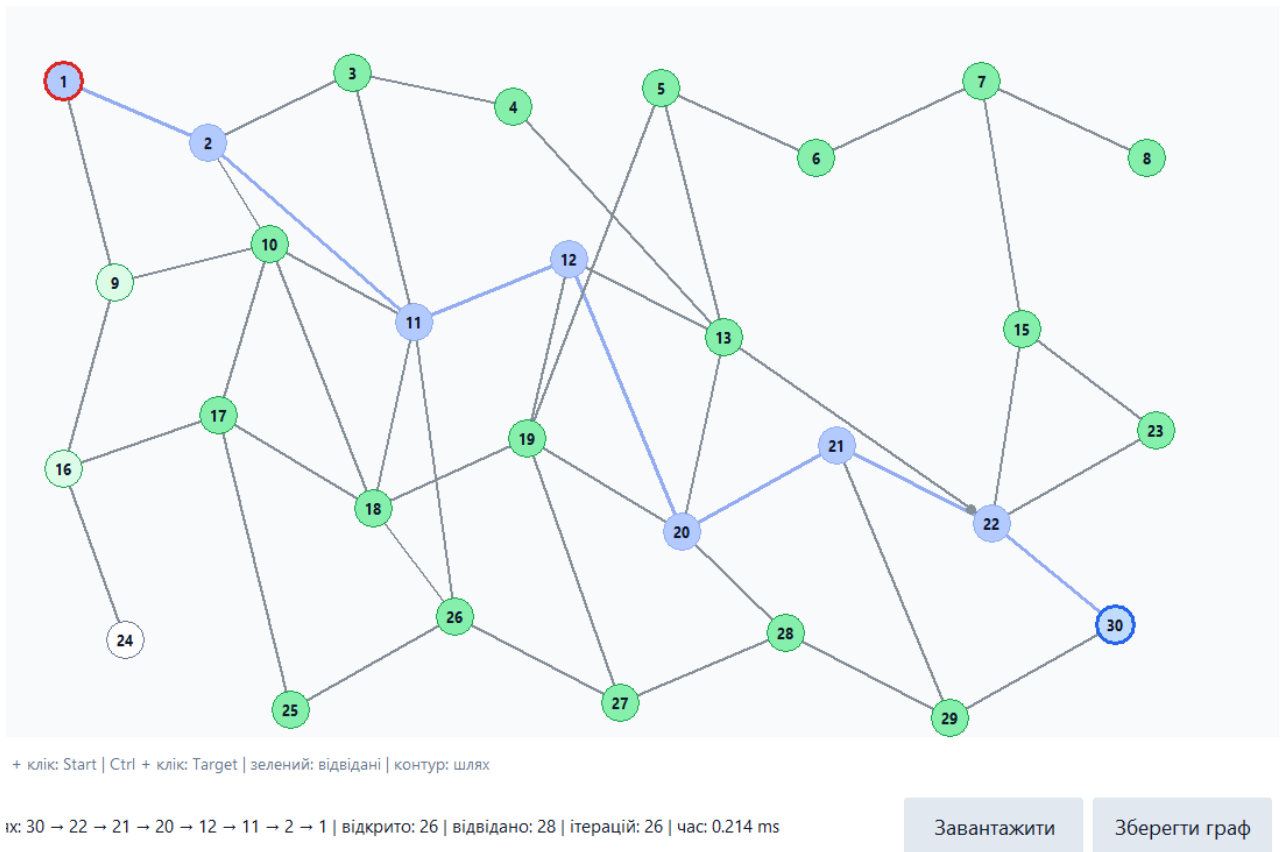


Рис.12 – Обхід неорієнтованого графа змінивши початку та кінцеві вершини місцями.

На рисунку 12 видно, що, змінивши початкову і кінцеву вершину місцями, отримали новий шлях.

Дослід 4: Використання пошуку вишир на різних видах графів.

Завантажимо орієнтований підготовлений граф та запустимо пошук при різних початкових та кінцевих вершинах.

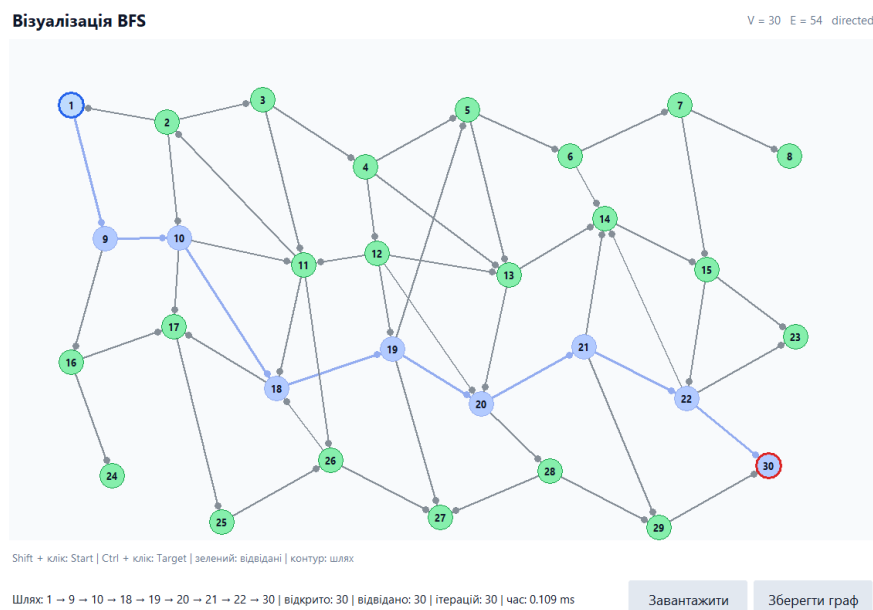


Рис.13 – Обхід орієнтованого графа з початковою вершиною 1 та кінцевою 30.

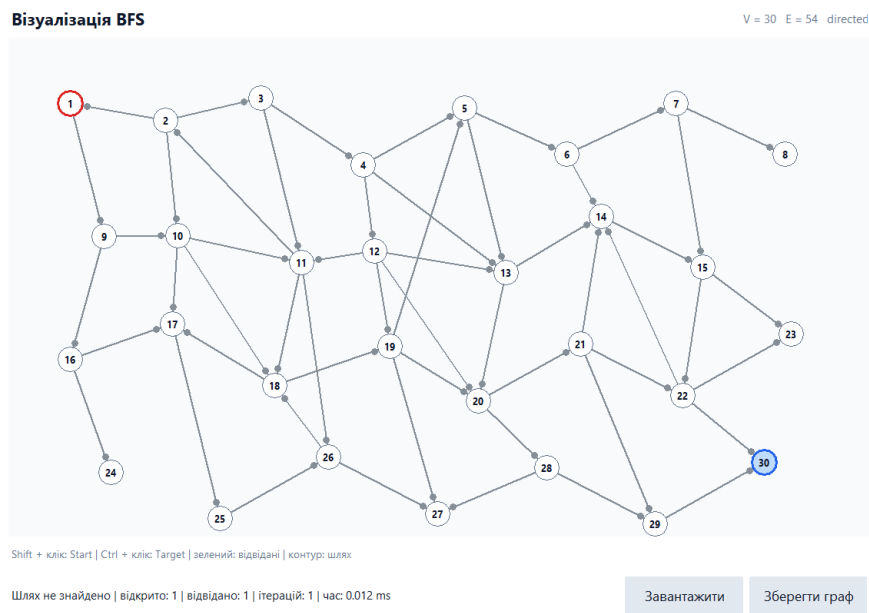


Рис.14 – Обхід орієнтованого графа з початковою вершиною 30 та кінцевою 1.

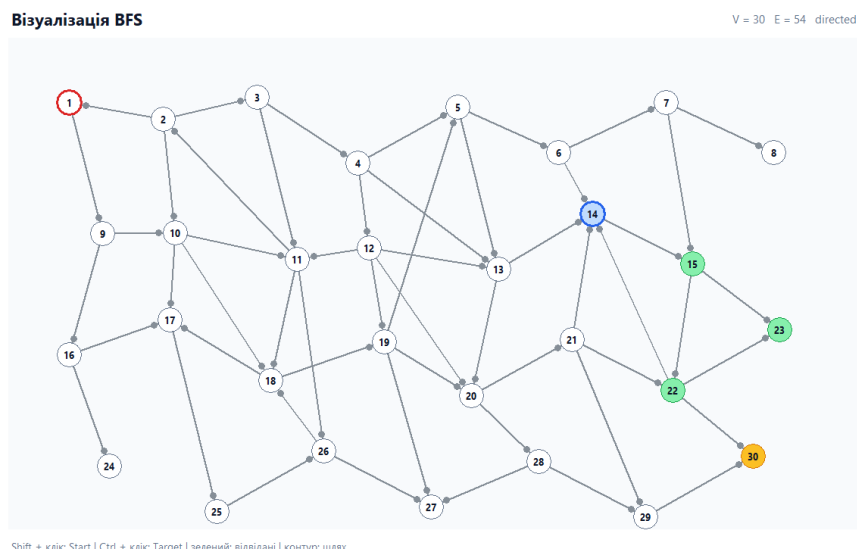


Рис.15 – Обхід орієнтованого графа з початковою вершиною 14 та кінцевою 1.

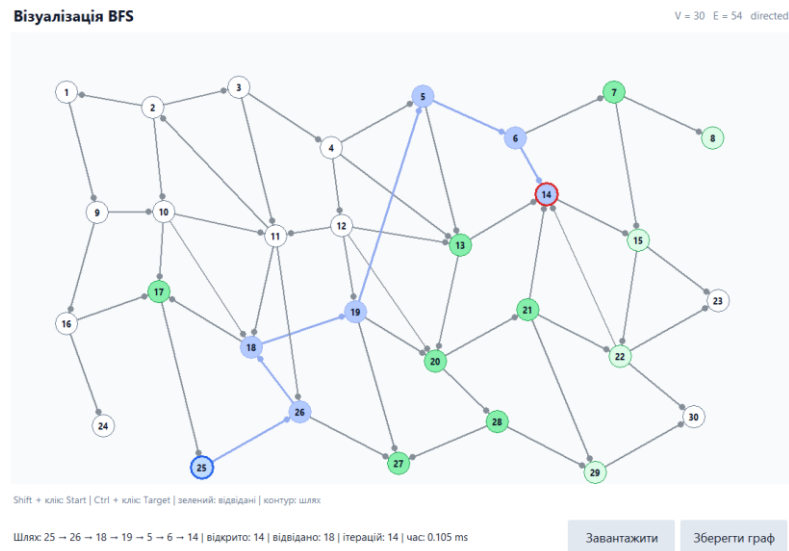


Рис.16 – Обхід орієнтованого графа з початковою вершиною 25 та кінцевою 14.

Як можна проаналізувати з рисунків 13-16, при різних початкових та кінцевих вершин, результуючий шлях можна знайти або його просто не існує.

Тепер використаємо граф-дерево та проведемо декілька експериментів із знаходженням шляху від однієї вершини до різних кінцевих вершин.

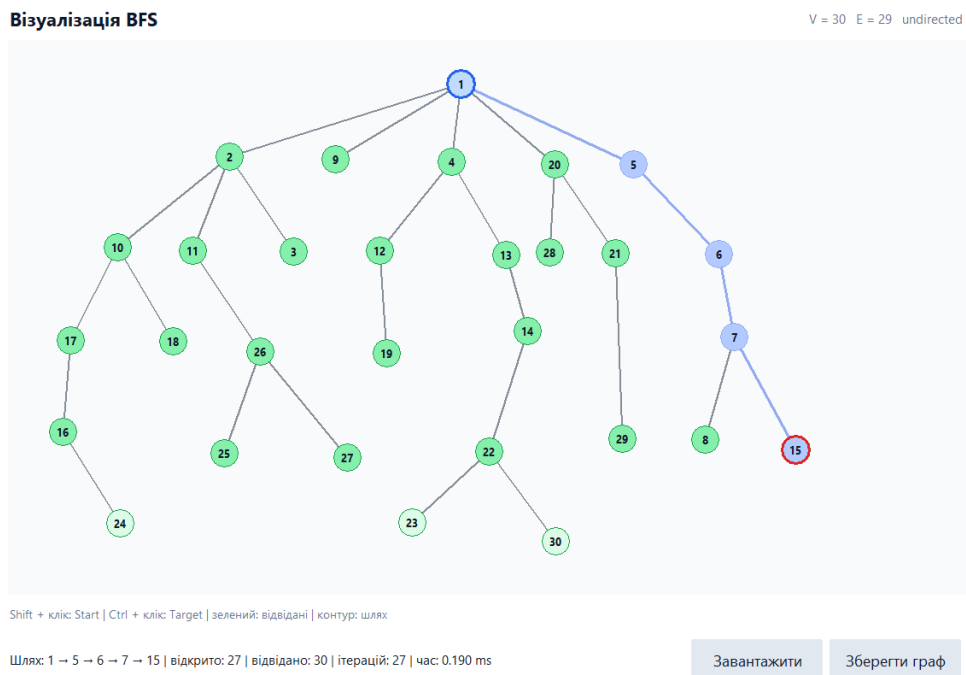
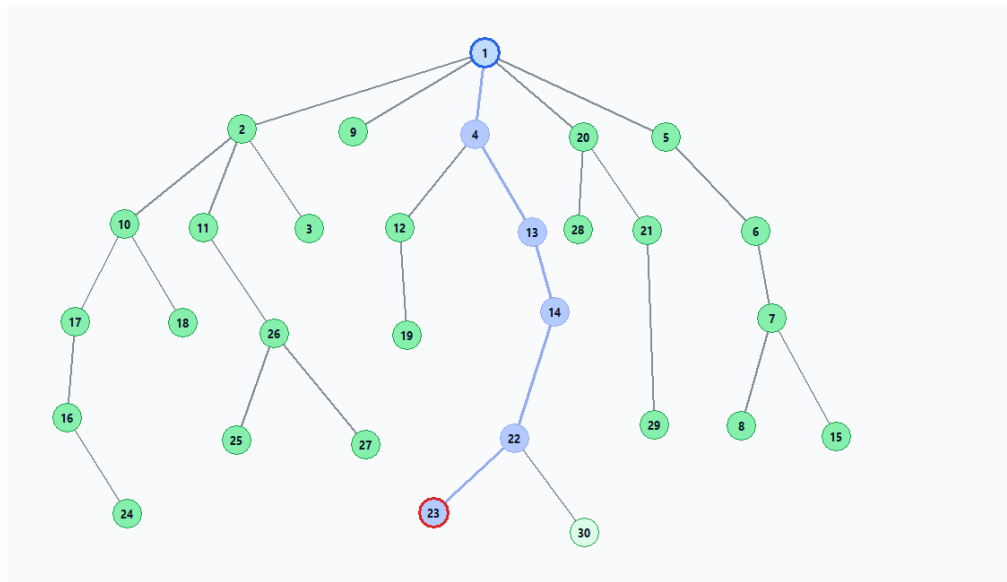


Рис.17 – Обхід дерева з початковою вершиною 1 та кінцевою 15.



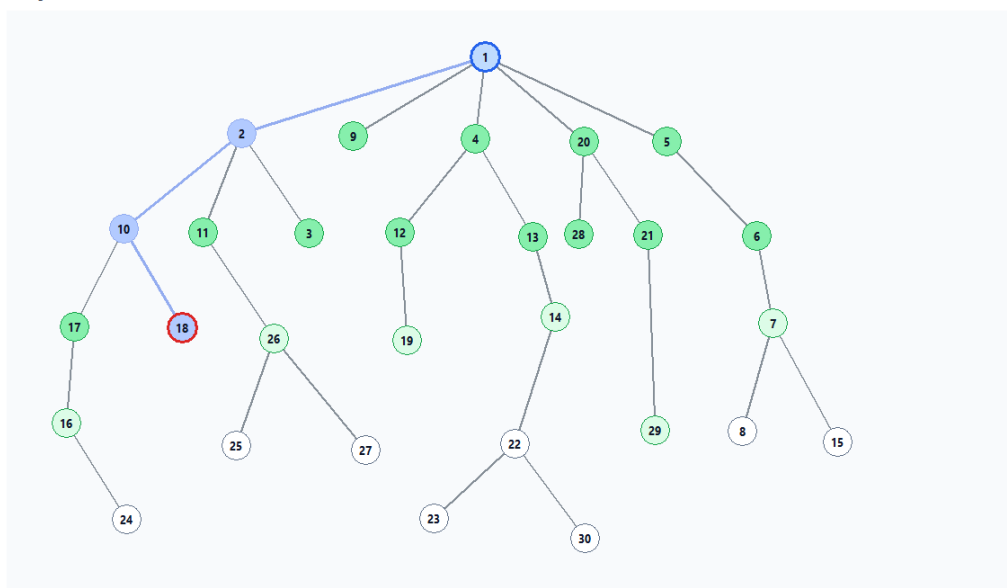
Shift + клік: Start | Ctrl + клік: Target | зелений: відвідані | контур: шлях

Шлях: 1 → 4 → 13 → 14 → 22 → 23 | відкрито: 29 | відвідано: 30 | ітерацій: 29 | час: 0.214 ms

Завантажити

Зберегти граф

Рис.18 – Обхід дерева з початковою вершиною 1 та кінцевою 23.



Shift + клік: Start | Ctrl + клік: Target | зелений: відвідані | контур: шлях

Шлях: 1 → 2 → 10 → 18 | відкрито: 16 | відвідано: 22 | ітерацій: 16 | час: 0.095 ms

Завантажити

Зберегти граф

Рис.19 – Обхід дерева з початковою вершиною 1 та кінцевою 18.

З рисунків 17-19 видно, як працює алгоритм пошуку вшир. Тобто різниця між вибору вершини на різних рівнях впливає на кількість відвіданих вершин на усіх кроках даного алгоритму.

Висновок

У ході виконання лабораторної роботи було розроблено програму реалізації пошуку в ширину (BFS) на графах та досліджено особливості її роботи за різних параметрів пошуку і конфігурацій графа. Розроблений графічний інтерфейс забезпечує візуалізацію графа та процесу пошуку, можливість зміни початкової і цільової вершин, порядку обходу суміжних вершин, редагування структури графа, а також відображення результатів виконання алгоритму.

У процесі експериментів було встановлено, що зміна порядку обходу суміжних вершин може впливати на послідовність розкриття вершин та на конкретний знайдений маршрут. Для досліджуваного неорієнтованого графа різні способи обходу дали відмінні результати, при цьому найкращим за кількістю виконаних операцій у проведеному експерименті виявився обхід за спаданням.

Зміна початкової та цільової вершин також вплинула на процес пошуку. Для неорієнтованого графа при зворотному напрямку пошуку було отримано інший порядок обходу та інші характеристики виконання алгоритму. Проведені експерименти показали доцільність дослідження обох напрямків пошуку для оцінювання кількості опрацьованих вершин і витрат пам'яті.

Під час зміни структури графа шляхом видалення та додавання вершин, ребер і дуг було встановлено, що навіть незначна зміна його структури може призвести до зміни результуючого маршруту та кількості опрацьованих вершин. Зокрема, після видалення вершин 4 і 14 маршрут змінився, а кількість необхідних ітерацій пошуку зменшилася. Після відновлення вершини 4 та додавання відповідних зв'язків алгоритм сформував новий маршрут, який використовував додану дугу.

Окремо було досліджено роботу BFS на орієнтованому графі та дереві. Для орієнтованого графа встановлено, що результат пошуку суттєво залежить від вибраних початкової та цільової вершин: у деяких випадках шлях існує, а в інших його знайти неможливо через напрямленість зв'язків. Для дерева проведені експерименти продемонстрували пошаровий принцип роботи BFS та залежність кількості відвіданих вершин від розташування цільової вершини на відповідному рівні пошуку.

Таким чином, поставлену мету лабораторної роботи досягнуто: реалізовано алгоритм пошуку в ширину та досліджено вплив порядку обходу, напрямку пошуку, структури графа і змін його елементів на результати роботи алгоритму. Проведені експерименти підтвердили, що BFS є ефективним алгоритмом для пошуку шляху в невагових графах, оскільки здійснює пошук пошарово та дозволяє знаходити найкоротший шлях за кількістю ребер. Водночас ефективність пошуку залежить від структури графа, напрямленості його зв'язків, вибраного порядку обходу та розташування цільової вершини.

Код програми

Код до даної лабораторної роботи розміщений у репозиторії https://github.com/MrCatman137/AI_systemem у директорії lab_1 та має повну інструкцію запуску даної програми.